

AI Project Scanner - Charter

1. Motivation / Problem

Modern AI/ML projects introduce security risks that extend beyond conventional application source code and dependencies. A single project may contain executable notebooks, model-loading code, third-party packages, serialized models, configuration, and externally sourced model artifacts, each with its own trust and supply-chain concerns.

Existing security tooling can address portions of this surface, but the security-relevant components of an AI/ML project are often analyzed independently. This makes it difficult to assess the project as a whole and to understand how its source code, dependencies, models, and provenance interact to create risk.

AI Project Scanner is intended to explore this problem: how can security tooling analyze the contents of an AI/ML project, identify meaningful risks across its software and model supply chains, and present evidence that developers and security engineers can act on?

2. Purpose & Objectives

AI Project Scanner will provide a security-focused view of an AI/ML project by identifying supported project components, applying analysis appropriate to each component, and reporting security-relevant findings with evidence and context.

- **Risk detection.** Detect meaningful security risks in AI/ML source code, notebooks, dependencies, models and serialized artifacts, configuration, and provenance information.
- **Safe analysis.** Analyze untrusted content safely, avoiding execution or deserialization during static analysis where doing so could introduce risk.
- **Explainable findings.** Produce findings that identify what was detected, why it matters, and the evidence supporting the finding.
- **Remediation.** Provide actionable remediation guidance where appropriate.
- **Automation.** Support scanning in software development and CI/CD workflows.
- **Extensible foundation.** Support additional AI/ML technologies, artifact formats, and security checks without requiring the initial implementation to cover the entire AI/ML ecosystem.

3. Scope & Non-Goals

AI Project Scanner focuses on the security-relevant contents of an individual AI/ML project and the software and model supply-chain relationships represented within it.

In Scope

- **Source code.** Application, training, inference, and model-loading code.
- **Notebooks.** Jupyter notebooks and other executable project content.
- **Dependencies.** Software dependencies and package manifests.
- **Models and artifacts.** Models, serialized objects, and other generated or acquired artifacts.

- **Configuration and metadata.** Information relevant to the security of the project.
- **Provenance and lineage.** Information associated with models, data, dependencies, and other project components.
- **Component relationships.** Relationships between project components when they provide useful security context.

The scanner may use both static and dynamic analysis. Static analysis should inspect untrusted content without executing it. Where runtime behavior is necessary to determine risk, dynamic analysis may execute or load content only within an intentionally isolated analysis environment.

Out of Scope

- **General-purpose scanning.** A source-code, dependency, or vulnerability scanner that duplicates mature existing security tooling without an AI/ML-specific reason.
- **Live infrastructure scanning.** A network reconnaissance or vulnerability scanner for deployed AI/ML services.
- **Detection and monitoring.** An intrusion-detection, SIEM, or runtime monitoring platform.
- **Governance and admission control.** A model registry, governance platform, policy engine, or admission-control system.
- **Complete AI/ML security coverage.** A solution for every stage of AI/ML security.

The scanner may produce information useful to those systems, but implementing them is outside the project's boundary.

4. Initial Approach

AI Project Scanner will analyze an AI/ML project as a collection of security-relevant components rather than treating the project as a single homogeneous input.

The scanner is expected to identify supported components and apply analysis appropriate to each type. Individual analyzers may inspect source code, notebooks, dependency metadata, model formats, serialized objects, configuration, or provenance information. Findings may also incorporate relationships between components when those relationships provide additional security context.

Static analysis will be preferred where it can answer the relevant security question safely and reliably. Untrusted models or serialized objects must not be loaded merely to inspect them. Where meaningful behavior cannot be determined statically, the project may introduce dynamic analysis using deliberately isolated execution environments with observable and restricted access to system resources.

Analysis results should ultimately use a consistent finding model that communicates the detected condition, supporting evidence, security impact or severity, relevant security-framework mappings where justified, and actionable remediation.

The initial implementation will intentionally support a small set of high-value analyses. Additional project types, model formats, analyzers, correlation capabilities, and dynamic-analysis techniques can be added incrementally as the design proves itself.

5. Guiding Principles & Constraints

- **Safe by default.** Analysis of potentially malicious or untrusted content must not unnecessarily expose the scanner or its environment to that content.
- **Evidence over assertions.** Findings should explain what was detected and provide sufficient evidence for a user to understand and investigate the result.
- **Actionable results.** Findings should help users understand both the risk and, where practical, how to address it.
- **Use existing security tooling where appropriate.** The project should complement or integrate established tools rather than reimplement mature capabilities without an AI/ML-specific reason.
- **Reproducibility and automation.** Analysis should be deterministic where practical and suitable for automated development and CI/CD workflows.
- **Extensibility without premature abstraction.** The design should accommodate additional analyzers and project components while avoiding infrastructure that is not justified by current requirements.
- **Least privilege.** Scanner components, automation, and any future dynamic-analysis environments should receive only the access required to perform their intended function.
- **Auditable development.** Significant architectural decisions and security-relevant changes should be visible through version-controlled documentation, review, and automated validation.

The project will initially favor a small, locally or CI-executable implementation with minimal operational overhead. Specialized cloud infrastructure or hardware should be introduced only when it enables a concrete capability.

6. Open Questions / Risks

Several questions should remain intentionally open at project inception:

- **Analysis coverage.** Which AI/ML project components and security checks provide the greatest value and should be prioritized beyond the initial implementation?
- **Finding model.** What common representation can describe findings across substantially different inputs while preserving useful evidence, severity, remediation, and security-framework context?
- **Project-level correlation.** How much additional security value can be obtained by relating findings across source code, dependencies, models, configuration, and provenance rather than analyzing each independently?
- **Dynamic-analysis isolation.** What level of sandboxing is sufficient to safely observe potentially malicious artifacts, and which runtime behaviors are worth capturing?
- **Provenance quality.** How should the scanner assess incomplete, inconsistent, or unverifiable provenance without implying a level of trust that the available evidence cannot support?
- **False positives and confidence.** How should findings communicate uncertainty and distinguish suspicious characteristics from demonstrated vulnerabilities or malicious behavior?
- **Extensibility.** How can additional analyzers and artifact formats be supported without introducing a plugin or abstraction model more complex than the project requires?

A broader project risk is **scope expansion**. AI/ML security spans application security, software supply chains, model security, data security, infrastructure, governance, and runtime behavior. The project must maintain a clear boundary and favor a small number of well-supported analyses over superficial coverage of the entire field.

7. Decision Evolution

This charter records the project's initial purpose, boundaries, principles, and technical direction at inception. It is intended to remain as a historical snapshot rather than being continuously revised to reflect the current architecture.

Significant architectural decisions made after inception will be recorded separately as Architecture Decision Records (ADRs). ADRs may be superseded as the project evolves, while preserving the reasoning and context behind earlier decisions.

Current behavior, supported capabilities, and usage will be documented separately in the project's README and other living documentation.